Week 3 Assignment Report

GPU Memory Hierarchy & Performance Optimization

Priyanshu Gonnade

Roll Number: 24B3959

January 26, 2026

1 Overview and Motivation

The objective of this assignment is to study the impact of GPU memory hierarchy on kernel performance. Unlike Week 2, which focused on basic kernel execution and parallelism, this week emphasizes how memory access patterns, shared memory usage, and bandwidth utilization influence execution speed.

The assignment is divided into three tasks:

- Analysis of coalesced vs non-coalesced global memory access
- Performance improvement using shared memory
- Establishing a CPU baseline for the final project workload

2 Task 1: Global Memory Access Patterns

2.1 Experiment Description

Two CUDA kernels were implemented to process a large 1D array of $N = 16{,}777{,}216$ floating-point elements.

- **Coalesced Kernel:** Thread i accesses element i
- **Non-Coalesced Kernel:** Thread i accesses element $i \times 32$

The computation performed by both kernels is identical; only the memory access pattern differs. Execution time was measured using CUDA events, and effective bandwidth was calculated.

2.2 Results

Table 1: Memory Coalescing Performance Comparison

Access Pattern	Stride	Time (ms)	Bandwidth (GB/s)
Coalesced	1	0.583	230.10
Non-Coalesced	32	0.565	7.43

2.3 Analysis

Although the execution times appear similar, the effective bandwidth differs drastically. In the non-coalesced case, each warp accesses memory locations that span multiple cache lines, leading to inefficient utilization of global memory bandwidth.

Conclusion: Strided access wastes nearly 97% of available bandwidth because the GPU loads full cache lines but uses only a single element per transaction.

3 Task 2: Shared Memory Optimization

3.1 Experiment Description

Matrix multiplication ($C = A \times B$) was implemented for matrix size $N = 2048$ using two approaches:

- **Naive Implementation:** All reads from global memory
- **Tiled Implementation:** 32×32 tiles loaded into shared memory

Shared memory allows reuse of matrix elements, significantly reducing global memory traffic.

3.2 Results

Table 2: Matrix Multiplication Performance

Implementation	Time (ms)	Speedup
Naive (Global Memory)	53.718	1.0x
Tiled (Shared Memory)	41.965	1.28x

3.3 Analysis

The shared memory version achieves a 1.28x speedup by reducing repeated global memory accesses. Further speedups are possible through improved tile sizing and occupancy tuning.

4 Task 3: CPU Baseline for Final Project

4.1 Workload Description

The final project focuses on an Edge AI NPU simulation. The core workload is a matrix-vector multiplication (GEMV) representing a neural network linear layer.

- Batch size: 1
- Input features: 4096
- Output features: 4096

4.2 Baseline Measurement

The workload was executed on the CPU using NumPy to establish a reference runtime.

Table 3: CPU Baseline Results

Parameter	Value
Input Dimensions	1×4096
Weight Matrix	4096×4096
CPU Runtime	4.567 ms

5 Conclusion

This assignment highlights that memory bandwidth is often the primary bottleneck in GPU programming. Properly coalesced memory access is essential for achieving peak performance, while shared memory provides significant speedups for data-reuse-heavy workloads.

The CPU baseline establishes a clear performance target for upcoming GPU optimizations in Weeks 4–6.